EventSync

Project technology and architecture guide

Stack | End-to-end flow | Role features | API | Data model

Executive overview

EventSync is a role-based event management web application. A React single-page client communicates with an ASP.NET Core Web API. The API applies business rules and uses Entity Framework Core to persist users, events, registrations, attendance, announcements and reviews in SQL Server. Several documents and ticket assets are generated in the browser.

At a glance: React 19 + Vite frontend -> HTTPS JSON API on ASP.NET Core / .NET 9 -> EF Core 9 -> SQL Server LocalDB. JWT Bearer tokens are used for API authentication.

Guide contents

- 1. Technology stack and package roles
- 2. Application flow from browser to database
- 3. Functionality by user role
- 4. Feature-to-library and API mapping
- 5. API routes and persistence model
- 6. Configuration, limitations and implementation notes

Repository context: EventManagement solution. Details below describe source code and configuration inspected in the repository; declared dependencies are distinguished from packages actually used in application code.

1. Technology stack

Frontend technologies

JavaScript / JSX Application language and React component syntax. The client is JavaScript, not TypeScript.

React 19 / ReactDOM Component-based UI; state and effects manage forms, tabs, loading states and API-backed dashboard data. Mounted from frontend/event/src/main.jsx.

Vite 8 + React plugin Frontend development server and production bundler. vite.config.js also configures a local API proxy for https://localhost:7165.

React Router 7 Client-side routes, navigation, query parameters and protected dashboard page selection.

Axios HTTP client for JSON API requests. Shared instance/interceptors and event, registration, attendance, announcement and review services are in src/services/api.js. Some dashboards make direct Axios calls.

React Toastify Success/error toast messages on authentication forms and a global ToastContainer.

CSS / SVG Custom stylesheets and inline CSS style the application. Icons include inline SVG paths; the source does not import Bootstrap styles or an icon package.

Frontend feature libraries

qrcode Creates QR-code data URLs locally for employee event passes.

jsPDF Creates ticket PDFs, certificate PDFs and organizer report PDFs in the browser.

html2canvas Captures the rendered certificate preview as an image for inclusion in a PDF.

Backend and persistence

.NET 9 / C# Backend runtime and implementation language. Target framework is net9.0.

ASP.NET Core Web API REST controllers, DTO validation, configuration, dependency injection, middleware, HTTPS redirection and CORS.

Entity Framework Core 9 ORM and SQL Server provider; DbContext, entity mappings, LINQ queries, persistence, indexes, relationships and migrations.

SQL Server LocalDB Configured development database (EventSyncDB), using Windows trusted connection.

JWT Bearer + PasswordHasher API token validation/claims and password hashing/verification.

Swagger / Swashbuckle API endpoint documentation and Swagger UI in development.

SendGrid SDK Email-sending code is present in AuthService; the current controller password-reset endpoint does not invoke that service.

Tooling and dependency notes

Frontend scripts: npm run dev, npm run build, npm run lint, npm run preview. ESLint with React Hooks and React Refresh plugins is configured as development tooling. Package versions are declared in frontend/event/package.json; backend NuGet references are in EventManagement/EventManagement.csproj.

- Bootstrap, Bootstrap Icons and React Icons are declared in the frontend package manifest, but no application-source imports were found. They appear unused in the current UI.
- generateCertificate.js contains a jsPDF helper but is not imported by the current certificate tab; the active tab has its own html2canvas + jsPDF path.
- CSV export is labeled ?Excel (CSV)?; it emits CSV text, not an XLSX workbook.

2. Application flow

Startup and navigation

- The browser loads frontend/event/index.html and starts src/main.jsx, which mounts the React app.
- App.jsx wraps the routes in BrowserRouter and AuthProvider and renders the global ToastContainer.
- AppRoutes.jsx maps /login, /register and /forgot-password to public pages; role-specific dashboards are wrapped in ProtectedRoute.
- ProtectedRoute checks token and user values in localStorage and redirects to login or the matching role dashboard when access does not match.

Authentication and API requests

- Login submits email, password and selected role to POST /api/auth/login. Registration submits name, email, password, department and an Employee or Organizer role to POST /api/auth/register.
- On login, the API verifies the password, checks the requested role, creates an eight-hour JWT with identity, role and department claims, and returns token plus user data.
- The frontend stores token and user data in localStorage; AuthContext normalizes the role and routes the user to their role dashboard.
- Axios attaches a Bearer token for API calls. A 401 response removes stale token/user entries; the API validates JWT signature, issuer, audience and expiry.
- The API controller validates inputs and applies role/department scoping where implemented, then uses ApplicationDbContext (EF Core) to query or save SQL Server records.
- JSON responses populate dashboard components. User actions such as registration, event edits, check-in, reviews and announcements send new API requests.

Browser-generated outputs

The frontend generates QR codes for tickets, creates ticket and certificate PDFs, and exports organizer reports as CSV or PDF. These operations are client-side; there is no separate report-generation service in the inspected code.

Conceptual request path: Browser UI -> React page/service -> Axios or fetch -> HTTPS /api/... -> ASP.NET Core controller -> EF Core DbContext -> SQL Server -> JSON response -> React state/UI.

Configuration

- API origin used by the shared client and dashboard code: https://localhost:7165. Vite proxy targets the same backend.
- CORS is configured for http://localhost:5173 and https://localhost:5173.
- SQL Server LocalDB connection and JWT settings are configured in appsettings.json. Treat secrets and development defaults as environment-specific configuration, not production-ready settings.

3. Functionality by role

Employee portal

Main implementation: frontend/event/src/pages/employee/EmployeeDashboard.jsx; UI tabs are split into employee/tabs/*.

- Dashboard: summary cards and upcoming-event information.
- Discover: browse events, search/filter and open the event registration form.
- My Events: review event registrations associated with the employee.
- Tickets: view pass details and QR code, download a ticket PDF, and submit check-in.
- Certificates: list checked-in registrations, preview a participation certificate, and download it as PDF.
- Reviews: retrieve checked-in events and submit a rating/comment when eligible.
- Notifications: display event announcements and update local read state; the UI also attempts a server read update.
- Profile: display information available from the signed-in user.

Organizer portal

Main implementation: frontend/event/src/pages/organizer/OrganizerDashboard.jsx. The dashboard loads events, registrations, attendance, announcements and reviews. It filters events and related data by the organizer department where possible.

- Dashboard metrics: department events, registrations, confirmed seats, check-ins, attendance rate and announcements.
- Events: create, edit, list and delete event records.
- Registrations: review registrations scoped to events in the organizer department.
- Attendance: inspect check-in records and toggle a check-in state.
- Announcements: create event-specific announcements.
- Reviews: view reviews for the organizer department.
- Reports: select attendee list or registration log and export CSV or PDF.

Admin portal

Main implementation: frontend/event/src/pages/admin/AdminDashboard.jsx. It fetches users, events, registrations, announcements and reviews for administrative summaries.

- System overview: account, event, registration and announcement metrics; review data is used for ratings.
- Event Schedules: calendar-style view with month navigation and event selection.
- Events: inspect events and filter/display by department.
- Announcements: review and create event announcements.
- User listing: loads account data through the admin-only /api/users endpoint and identifies organizer records.

Roles: Admin, Employee and Organizer are represented by UserRole enum values (Admin = 1, Employee = 2, Organizer = 3). Public registration only accepts Employee or Organizer; Admin registration is explicitly rejected by the API.

4. Feature-to-library and API mapping

What powers each feature

Routes and role navigation React Router: BrowserRouter, Routes/Route, Navigate, useNavigate and useSearchParams. ProtectedRoute applies client-side token/user/role checks.

Login and registration React state for form state; authService.js + Axios for POST requests; ASP.NET DTO validation and PasswordHasher/JWT logic on the backend; React Toastify for user feedback.

Dashboard data Axios shared service methods or direct Axios calls; React useEffect/useState load and render records. API endpoints provide persistent data via EF Core.

Employee QR ticket qrcode generates a PNG data URL from a payload composed from ticket code, event title and attendee email; React renders it and jsPDF embeds it into the downloaded pass.

Ticket PDF jsPDF creates a one-page pass with event, ticket, date, venue and attendee details; QR image is embedded when available.

Certificate PDF CertificatesTab fetches registrations, filters for checked-in records, renders an HTML certificate preview, html2canvas captures it, then jsPDF places the captured image into a landscape A4 PDF.

Organizer report CSV exportEventReport.js builds registration/attendance rows, escapes CSV cells and protects spreadsheet formula-leading values, then downloads a .csv file with a UTF-8 BOM.

Organizer report PDF The same report utility uses jsPDF to lay out attendee or registration columns across landscape A4 pages.

Reviews ReviewsTab uses reviewService methods; API checks employee identity and check-in eligibility, validates 1-5 rating and prevents duplicate event/employee reviews.

Announcements API endpoints return/create/delete announcement records. Organizer and admin creation checks user role; organizers are restricted to their event department.

Styling and icons Project CSS and inline styles; inline SVG icons. Bootstrap/Bootstrap Icons/React Icons are dependencies but not imported by source code found.

Frontend service endpoints

eventService GET /api/events, GET /api/events/{id}, POST /api/events, PUT /api/events/{id}, DELETE /api/events/{id}.

registrationService GET /api/registrations, POST /api/registrations, DELETE /api/registrations/{id} (the controller currently shown has no DELETE action).

attendanceService GET /api/attendance, POST /api/attendance/check-in/{eventId}, PUT /api/attendance/{id}/toggle.

announcementService GET /api/announcements and POST /api/announcements; the client service does not define delete, though the API has DELETE.

reviewService GET /api/reviews, GET /api/reviews/checked-in-events, POST /api/reviews.

authService POST /api/auth/register, POST /api/auth/login and POST /api/auth/forgot-password.

5. API routes and data model

Backend route summary

AuthController /api/auth POST register; POST login; POST forgot-password. Authentication responses contain JWT and user/role details.

EventsController /api/events GET list (optional department filter); GET by ID; POST create; PUT update; DELETE.

RegistrationsController /api/registrations GET role-scoped registrations; POST registration; POST {id}/check-in. The API checks the referenced event, creates a registration and increments the event registered count.

AttendanceController /api/attendance GET role-scoped attendance; POST check-in/{eventId}; PUT {id}/toggle; POST and DELETE attendance records.

AnnouncementsController /api/announcements GET ordered announcements; POST create (Admin or Organizer, with organizer department check); DELETE {id} (Admin).

EventReviewsController /api/reviews GET reviews (Admin/Organizer); GET checked-in-events (Employee); POST review (Employee, checked-in event required, one review per event).

UsersController /api/users GET user list; API restricts this endpoint to Admin.

Persistent entities

User Full name, unique email, password hash, department, role and creation time.

Event Title, description, date, location, department, capacity, registered count, type, status and image URL.

Registration Attendee name/email, event ID, tier, registration date, status, checked-in flag and check-in timestamp.

Attendance Event ID, attendee name, ticket code, checked-in flag and time.

Announcement Title, event ID, timestamp and body.

EventReview Employee and event details, rating, optional comment and creation time.

Relationships and constraints

- User email has a unique index.
- Review has a unique composite index on EventId and EmployeeEmail.
- Registrations, attendance, announcements and event reviews reference events; event deletion is configured to cascade to these related records.
- EF Core migrations define the initial tables and the later EventReviews table/index.

Core persistence implementation: EventManagement/Data/ApplicationDbContext.cs. Entity definitions are under EventManagement/Models, with Event and Registration in the EventSync.Models namespace.

6. Configuration and implementation notes

Observed limitations and integration gaps

- Password reset: the frontend says it sends a reset link, but AuthController currently returns a message without calling AuthService. AuthService contains SendGrid email code; the controller does not use it. No reset-password frontend route was found, and the generated reset token in the service is not persisted in the inspected implementation.
- Notification read state: EmployeeDashboard calls PUT /api/announcements/{id}/read, but the current AnnouncementsController implements GET, POST and DELETE only. The client still updates local state before the attempted server request.
- Authorization coverage: registrations, announcements, reviews and users controllers explicitly use [Authorize]. Events and attendance controllers do not declare [Authorize] in the inspected code. Client-side ProtectedRoute checks do not replace server-side API authorization.
- Role-specific filtering: registrations and attendance apply role-based filters; reviews check roles and employee check-in eligibility; announcements enforce role and organizer department for creation. These policies are not uniform across all controllers/actions.
- API configuration is split: shared Axios uses a fixed https://localhost:7165 base URL; dashboards also make direct calls; CertificatesTab reads VITE_API_URL with a localhost fallback. Vite proxy is configured, but most observed requests use absolute URLs.
- Forgot-password responses: the current controller returns a success-shaped message for an unknown email and for a known email, but does not send the email itself.
- No payment amounts appear in the model or report. Ticket tiers exist, but pricing/payment totals are not stored.
- frontend/event/README.md is the default React/Vite template, not project-specific setup documentation.

Important configuration files

frontend/event/package.json Frontend dependencies and npm scripts.

frontend/event/vite.config.js React plugin and local API proxy.

EventManagement/EventManagement.csproj .NET target framework and NuGet package references.

EventManagement/Program.cs Database, JWT authentication, CORS, Swagger and HTTP pipeline setup.

EventManagement/appsettings.json LocalDB connection, JWT issuer/audience/key and SendGrid settings. Do not use development secrets as production credentials.

EventManagement/Migrations EF Core database schema history.

Source map

- Frontend entry and routing: frontend/event/src/main.jsx, App.jsx, routes/AppRoutes.jsx, routes/ProtectedRoute.jsx.
- Authentication state and HTTP: frontend/event/src/context/AuthContext.jsx, services/authService.js, services/api.js.
- Role screens: frontend/event/src/pages/admin, organizer and employee.
- Backend controllers: EventManagement/Controllers; entities: EventManagement/Models and EventSync.Models; DbContext: EventManagement/Data/ApplicationDbContext.cs.

This guide reflects the repository as inspected. Some features are present in UI code but are not fully wired to a matching backend endpoint; see the integration gaps above.